Computer Vision - Mid Project Report

Alessio Marchetti, Sirio Trentin, Nicolò Spuri

April 28, 2026

1 Introduction

This project implements a system for detecting and localizing moving objects in image sequences. The objective is to output a 2D bounding box representing the coordinates of the detected dynamic object in the first frame of the sequence of images. The solution makes use of techniques to identify and track features over time.

2 Instructions for Use

To build and run the project:

1. add the images and labels folders in the `resources` folder, and diveded respectively in the `data` and `labels` folder;
2. open the terminal in `build` path and run `cmake ..`, `make`;
3. finally run `./bin/main` to execute the project;
4. you can also run `./bin/main [folder name]` to execute on a specific images folder.

When running the execution command, the program will process all categories in the `resources/data` folder, compare the generated predictions against the ground truth labels, and automatically generate the results to be viewed in this report.

3 Approach

The solution combines traditional computer vision techniques, such as feature extraction (like ORB, SIFT, or KAZE) and optical flow, to differentiate between background motion and dynamic object motion.
We started by trying different approaches for the task, in particular:

- Nicolò Spuri tested optical flow with a simple feature detector;
- Alessio Marchetti and Sirio Trentin tried to manually track features from one frame to the next, matching feature descriptors.

After several tests, we concluded that the best-performing feature detector is ORB.
Then, Alessio Marchetti proposed and implemented a hybrid solution, combining the two previous approaches. A change was made in `evaluate_performance.cpp`, where Nicolò Spuri implemented Optical Flow to track features from the current frame to the next. It works as follows:

1. every `hybridApproachRate` iterations, the `updateTrackedKeypoints` method is called, and *minMotion* is increased, in order to track only relevant features;

2. inside the method, the features that survived the previous optical flow iterations are saved in `verifiedFirstFrameKP`;

3. the current frame's features and descriptors are recomputed;

4. a matching operation is performed between the initial and current frame descriptors;

5. the matched features are returned, and optical flow is applied to them.

This serves both to save keypoints after they have survived for some iterations, and to find new keypoints if they were initially stationary. Finally, Sirio Trentin implemented the testing of the project in `evaluate_performance.cpp` and modified the algorithm to dynamically adapt the `minMotion` distance threshold if the number of keypoints decreases too quickly during the optical flow iterations.

4 Results

The project performance on the target dataset (including bird, car, frog, sheep, and squirrel sequences) is evaluated using Mean Intersection over Union (mIoU) and Detection Accuracy (IoU > 0.5).

The automatic results derived from executing the evaluation are included below:

Category	**IoU**
car	0.7983
sheep	0.6711
bird	0.5073
frog	0.7492
squirrel	0.0832

Table 1: IoU per category.

Metric	**Value**
Mean IoU	0.5618
Accuracy	0.8000 (4/5)

Table 2: Overall Performance Metrics.

Initial Frame Tracking Results

car

sheep

bird

frog

squirrel

Figure 1: Initial Frame for each category with tracked bounding box.